THE ARCHITECT'S FIELD NOTES

BEHAVIORAL INTERVIEW STUDY GUIDE / 04 OF 07

Credit Card Authorization System — 10-Minute System Design Answer

THINK CLEARLY. DECIDE WITH EVIDENCE. DELIVER WITH CONFIDENCE.

Evidence-based STAR answers, senior-level judgment, and focused interview rehearsal.

PREPARED FOR SENIOR / PRINCIPAL SOLUTION ARCHITECT INTERVIEWS

READ · REFLECT · PRACTICE

04



YOUR STUDY COMPANION

Role focus: Solution Architect **Scope:** Issuer-side credit card authorization **Focus:** Key architectural decisions, reasons, failure handling, and points to remember

NAVIGATION

Inside this guide

01 1. Problem and requirements

02 2. High-level architecture

03 3. Key architectural decisions and reasons

04 4. End-to-end flow

05 5. Failure handling

06 6. Authorization vs. clearing vs. settlement

07 7. Key points to remember

08 8. Interview closing statement

CHAPTER

1. Problem and requirements

Design a system that receives a payment-network authorization request and quickly returns **Approve** or **Decline**. It must validate the card, assess fraud risk, check available credit, and durably reserve credit if approved.

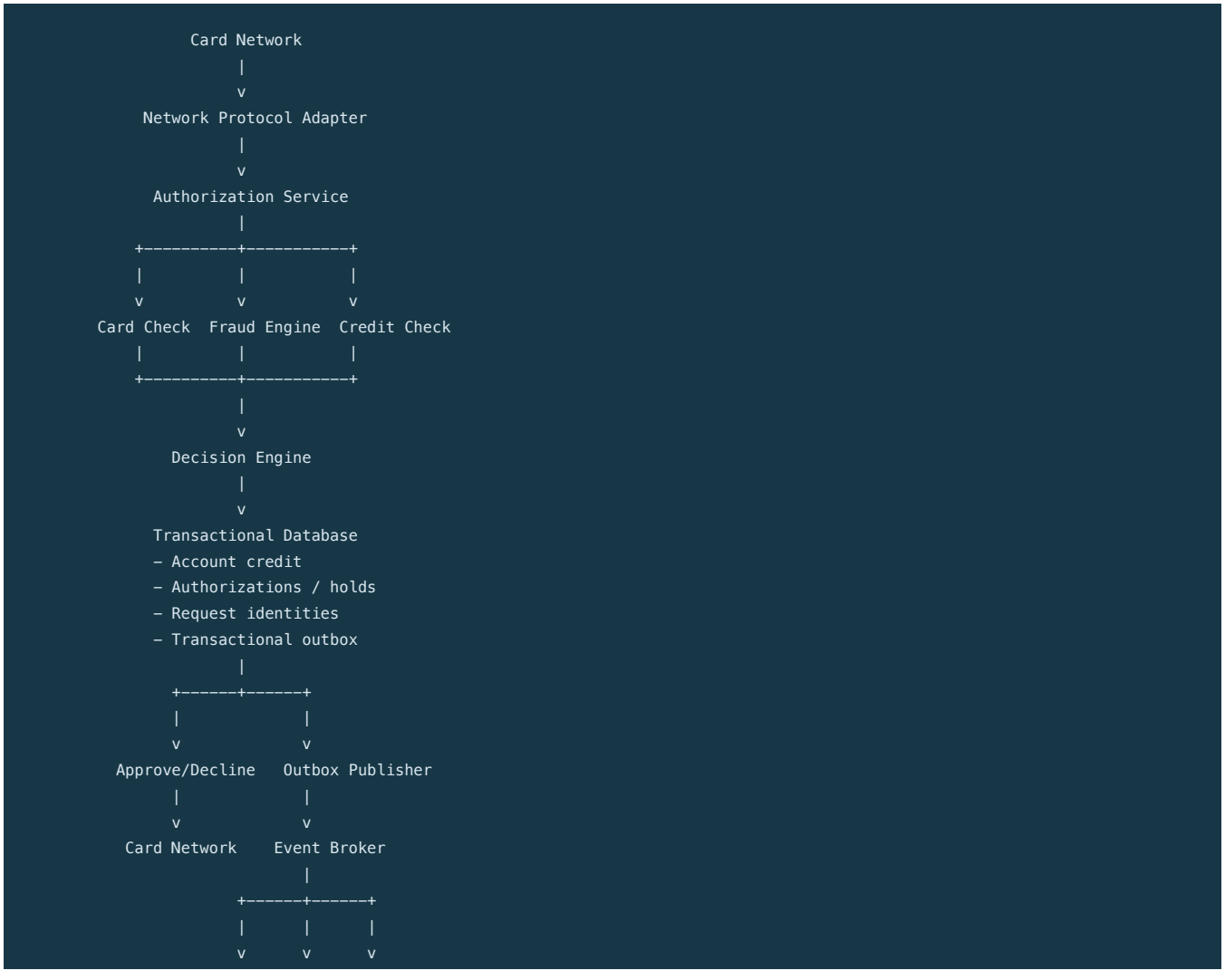
Key non-functional requirements

- **Low latency:** The merchant needs an immediate response.
- **High availability:** Component failures should not unnecessarily stop purchases.
- **Strong consistency for credit reservation:** Prevent unintended credit overcommitment.
- **Idempotency:** A retried authorization must not create a duplicate hold.
- **Security:** Protect card credentials, sensitive data, and cryptographic operations.

Illustrative scale assumption: 5,000–10,000 peak authorization requests per second. This is a design exercise assumption, not an American Express production figure.

CHAPTER

2. High-level architecture



```
                    Alerts  BI    Fraud updates
```

Core idea: Keep the **approval path synchronous**; move noncritical downstream work to **durable asynchronous processing**. Logical boxes do not necessarily require separate microservices.

CHAPTER

3. Key architectural decisions and reasons

Decision	Choice	Reason
Network integration	Protocol adapter and canonical internal request	Isolates network-specific message formats from business logic
Authorization	Synchronous request/response	The merchant requires a decision now
Authoritative storage	Transactional relational database	ACID transactions, constraints, and coordinated writes
Credit hold	Atomic conditional update	Prevents concurrent approvals from consuming the same available credit
Duplicate protection	Durable request identity and unique constraint	Safe retries without duplicate holds
Fraud decision	Synchronous required checks using rules and ML	Evaluate risk **before** approval
Fraud features	Low-latency feature store where suitable	Avoid expensive analytical queries on the critical path
Caching	Only for appropriately stale-tolerant information	Cached available credit is not authoritative
Async integration	Durable event broker, e.g., Kafka	Decouple notifications, analytics, and supporting updates
Event correctness	Transactional outbox + idempotent consumers	Avoid lost events after DB commit; tolerate redelivery
Scaling	Stateless workers; partition by credit account if necessary	Independent application scale; coordinated same-account writes
Multi-region	Single authoritative writer per credit account	Avoid conflicting reservations and split-brain approvals

A. Why a relational database?

Credit authorization requires a single transaction that:

1. Establishes the identity of the logical request.
2. Atomically checks and reserves available credit.
3. Saves the authorization and hold.
4. Saves the response decision and outbox event.

A relational database is a good baseline because it offers transactions, unique constraints, and concurrency controls. Other databases can work if they supply equivalent guarantees.

B. How do we prevent concurrent overcommitment?

Example: Available credit is $100. Two distinct $80 authorizations arrive together.

```
Without coordination:
Request A reads $100       Request B reads $100
A approves $80             B approves $80
Total reserved = $160       WRONG
```

Solution: An atomic, conditional update against the authoritative account state:

```
UPDATE account_credit
SET reserved_minor = reserved_minor + :amount
WHERE account_id = :account_id
  AND credit_limit_minor − posted_minor − reserved_minor >= :amount;
```

The database coordinates conflicting writes. Only one of the $80 requests can reserve credit under this example's rules. Create the corresponding authorization and hold **in the same transaction**.

Remember: Checking credit and reserving it must be one atomic operation, not a read followed by an unrelated write.

C. How do we handle duplicate network requests?

An issuer may commit an approval, but the network response may be lost. The network retries.

- Derive a stable request identity using the applicable network-defined transaction identifiers.
- Persist that identity with a **unique constraint**.
- If the same logical request returns, recover its prior decision; do not add another hold.
- Compare a canonical request fingerprint so a conflicting request cannot silently reuse the identity.
- Treat legitimate incremental authorizations and reversals as separate operations.

Remember: A timeout does **not** prove the original authorization failed.

D. Why synchronous fraud detection?

If a risk check is mandatory for approval, it must happen before the decision. Use rules and low-latency model inference with suitable online features such as recent spending velocity.

If the risk dependency fails, follow an explicit issuer-approved degraded-operation policy. Never assume it is safe to approve everything or decline everything. Hard fraud or account limits may require authoritative coordination, not stale feature-cache values.

E. Why a broker and transactional outbox?

Notifications and analytics should not block checkout.

```
Commit authorization + outbox event in one DB transaction
                        |
                        v
                  Outbox publisher
                        |
                        v
                    Event broker
                        |
                +-------+------+
                |              |
                v              v
            Notifications   Analytics
```

Without an outbox, a crash after committing the authorization but before publishing the event can permanently lose a downstream update. Consumers must be idempotent because the publisher may resend events.

F. How do we scale and make it available?

- **Stateless authorization workers:** Horizontal scale behind resilient network ingress.
- **Credit database:** Optimize indexes and transactions first; partition by the **underlying credit account**, not individual card, if scale demands it.
- **Risk services:** Independently scale feature reads, rules, and model serving.
- **Availability:** Use redundant instances and durable database replication with tested failover.
- **Multi-region:** Prevent simultaneous conflicting writers for one credit account; use ownership/fencing and verified recovery procedures.

CHAPTER

4. End-to-end flow

```
1. Receive payment-network authorization request
           |
2. Validate message; derive stable request identity
           |
3. Detect duplicate and recover prior decision if present
           |
4. Validate card/account and required authentication
           |
5. Evaluate fraud/risk policy
           |
6. Begin short DB transaction
           |
7. Atomically reserve available credit
           |
8. Save authorization, hold, response identity, outbox
           |
9. Commit transaction
           |
10. Return approve/decline via network adapter
           |
11. Deliver downstream events asynchronously
```

Critical detail: Do not hold a financial database lock while waiting for slow external risk calls. Recheck essential account/card conditions before commit when they could have changed.

CHAPTER

5. Failure handling

Failure	Handling
Duplicate authorization	Return previous decision; no extra hold
Concurrent purchases	Atomic credit reservation serializes conflicting account updates
Database crash before commit	Roll back; no normal approval

Committed approval but response lost	Retrieve stored decision on retry; apply network recovery/reversal rules
Fraud service unavailable	Use explicitly approved degradation policy
Broker unavailable	Retain durable event in outbox until publisher recovers
Duplicate reversal	Release only the eligible remaining hold, once
Late or partial clearing	Match to authorization; post exposure and adjust holds under explicit rules
Region failover	Ensure old primary is fenced before new ownership begins

CHAPTER

6. Authorization vs. clearing vs. settlement

- **Authorization:** Decide whether to approve and, if approved, reserve credit.
- **Clearing:** Process/present the financial transaction and post the corresponding obligation.
- **Settlement:** Complete the applicable movement of funds between participating institutions.

Example: A $100 authorized hold becomes a $100 posted obligation. To avoid counting the same amount twice, the corresponding hold must be released/consumed as the posting is recorded. Partial or mismatched clearing requires explicit matching and adjustment rules.

CHAPTER

7. Key points to remember

1. **Correctness first:** Credit check and reservation must be atomic.
2. **Idempotency is non-negotiable:** Retried requests must not create duplicate holds.
3. **Critical path is synchronous:** Required card, risk, and credit checks precede approval.
4. **Downstream work is asynchronous:** Notifications and analytics should not block checkout.
5. **Authoritative credit cannot come from a stale cache.**
6. **Timeout ≠ failure:** Persist decisions and handle uncertain network outcomes.
7. **Authorization ≠ settlement:** Holds and posted obligations are different financial states.
8. **Shard on the credit-account boundary:** Multiple cards may share one credit line.
9. **Single-writer account ownership simplifies correctness** during multi-region operation.
10. **Security is architectural:** Sensitive card data and cryptography need protected boundaries.

CHAPTER

8. Interview closing statement

> My design prioritizes financial correctness and low latency. I use synchronous processing for card validation, fraud decisions, and atomic credit reservation. A relational database provides transactional consistency, while durable idempotency prevents duplicate holds. I publish noncritical downstream events using a transactional outbox and a durable broker. For scale, I horizontally scale stateless services and partition authoritative state by credit account when required. For resilience, I use redundant services, controlled failover, and single-writer account ownership to prevent conflicting reservations.

Note: Throughput figures, latency targets, and component choices here are illustrative design assumptions, not verified internal American Express systems or requirements. Actual network response semantics, authentication rules, and recovery workflows depend on the applicable network agreements and product rules.